SENG 201
Software Engineering Foundations

[Week 3, Monday]



Last week...

- Java!
- A first overview of a Java program
- Inside a single Java file: data types, variables, and methods
- Java objects: `this`, constructors, fields, access modifiers

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

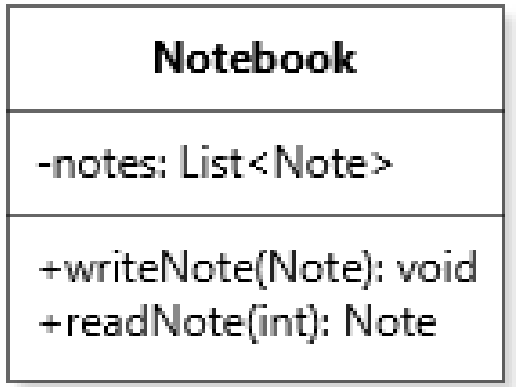
Last week...

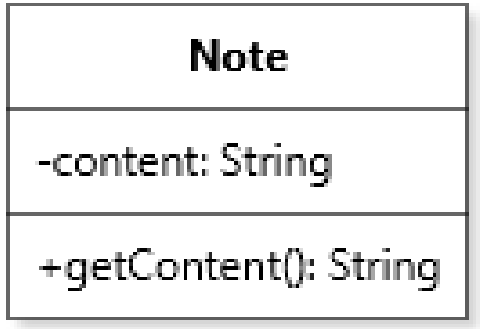
```java
public class Notebook {

    private final List<Note> notes = new ArrayList<>();

    public void writeNote(Note note) {
        notes.add(note);
    }

    public Note readNote(int index) {
        return notes.get(index);
    }
```





© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

A recent addition...

```
public String readAllNotes() {
    String fullContent = "Notebook\n--------";
    for (Note note : notes) {
        fullContent += "\n" + note.getContent();
    }
    return fullContent;
}
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Static and instance members

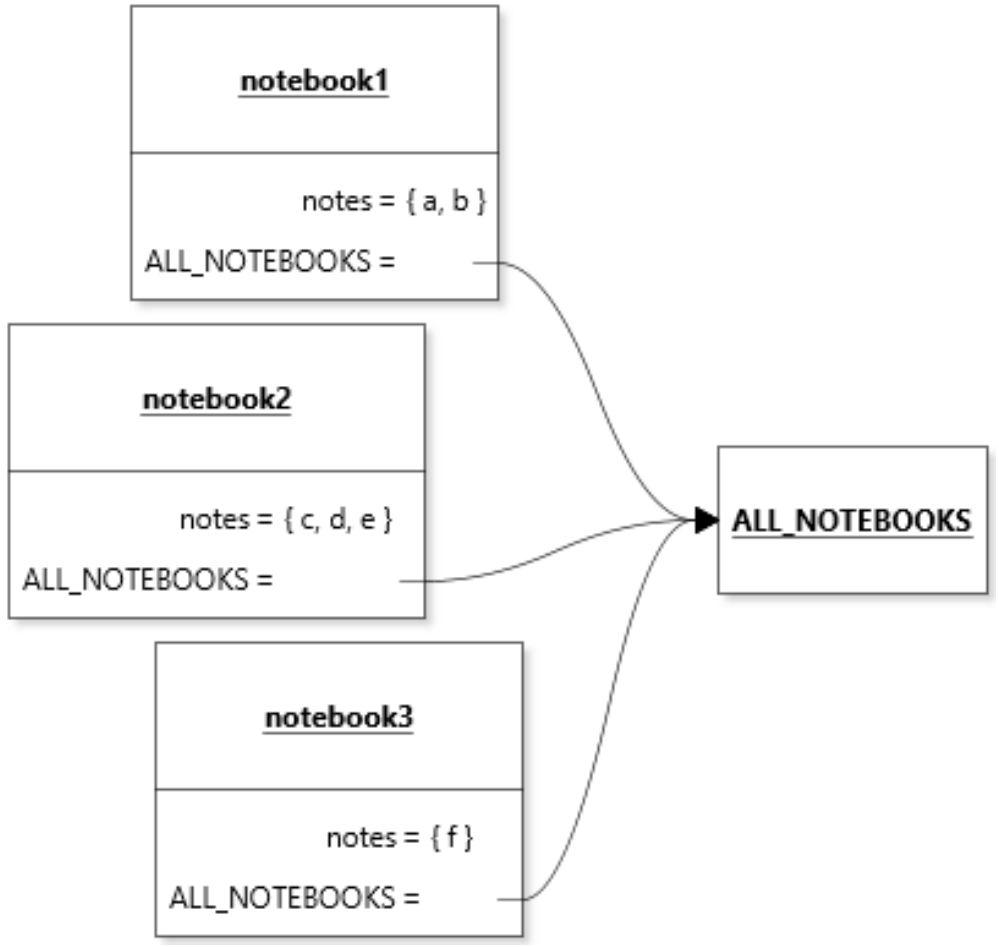
```java
private static final List<NotebookV4> ALL_NOTEBOOKS = new ArrayList<>();

public NotebookV4() {
    ALL_NOTEBOOKS.add(this);
}

public static int getCount() {
    return ALL_NOTEBOOKS.size();
}
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Static and instance members



Instance member

- each object has its own copy
- access from an object

Static member

- all objects have the same reference
- access from the class

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Static and instance members

```java
public static void addNoteToAllNotebooks(Note note) {
    notes.add(note);
}
```

can't access instance member in static method

```java
public static void addNoteToAllNotebooks(Note note) {
    for (NotebookV4 notebook : ALL_NOTEBOOKS) {
        notebook.notes.add(note);
    }
}
```

... unless on an explicit instance

```java
notebook.addNoteToAllNotebooks(new Note("common note"));
```

can access static member from instance

```java
NotebookV4.addNoteToAllNotebooks(new Note("common note"));
```

... but better to access from the class

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

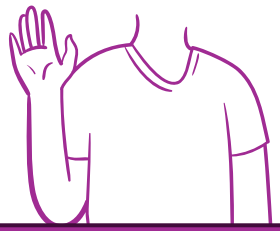
A solution to overloaded constructors

```java
public Note(String content) {
    this.content = content;
}

public Note(String... paragraphs) {
    this.content = String.join("\n", paragraphs);
}

public Note() {
    this.content = "<Empty note>";
}
```

```java
Note emptyOrVararg = new Note();
Note varargOrNo = new Note("1 paragraph");
```



How can we avoid overloading constructors?

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

A solution to overloaded constructors

```java
private Note(String content) {
    this.content = content;
}

public static Note createNote(String content) {
    return new Note(content);
}

public static Note createMultiNote(String... paragraphs) {
    return new Note(String.join("\n", paragraphs));
}

public static Note createEmptyNote() {
    return new Note("<Empty note>");
}
```

```java
Note emptyOrVararg = Note.createEmptyNote();
Note varargOrNo = Note.createMultiNote("1 paragraph");
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Multi-class Design



- → Packages
- → Dependencies in UML Class Diagrams

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Java Packages

Packages define a namespace in Java

Package names

- Components separated by '.' (e.g., `seng201.lecture3_1`)
- Fully qualified name (FQN) of a class is the package + class name
- Follows filesystem folder structure

class `nz.co.canterbury.seng201.notebook.Notebook`
is defined in
file nz/co/canterbury/seng201/notebook/Notebook.java

Classes in other packages must use import or FQN

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Java Packages

```java
package lecture2_3.notebook;
```

```java
import java.util.ArrayList;
//import java.util.List;

public class NotebookV4 {

    private static final java.util.List<NotebookV4> ALL_NOTEBOOKS = new ArrayList<>();
```

Fully qualified name (FQN)

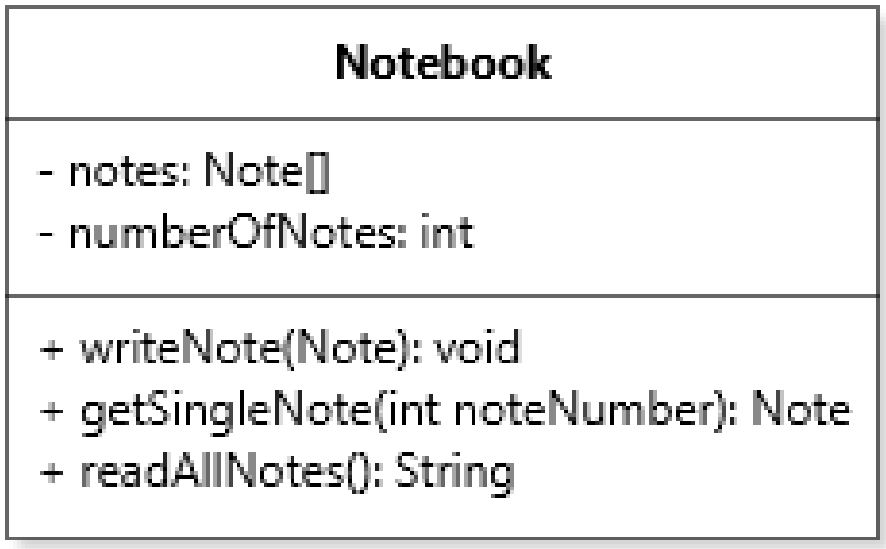
© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

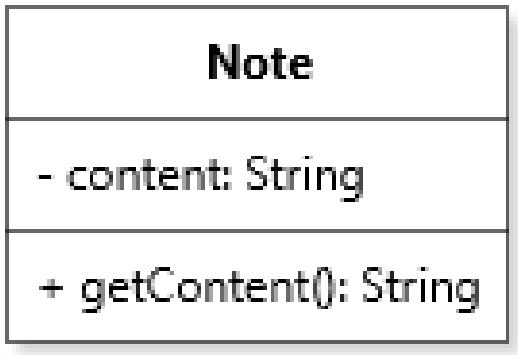
Let’s revisit UML class diagrams…

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

UML Class Diagrams

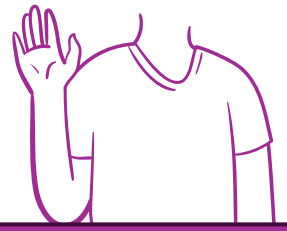
How can we indicate that the two classes depend on each other?





© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

UML Class Diagrams



What relationships exist between these concepts?

Vehicle

Driver

Car

Wheel

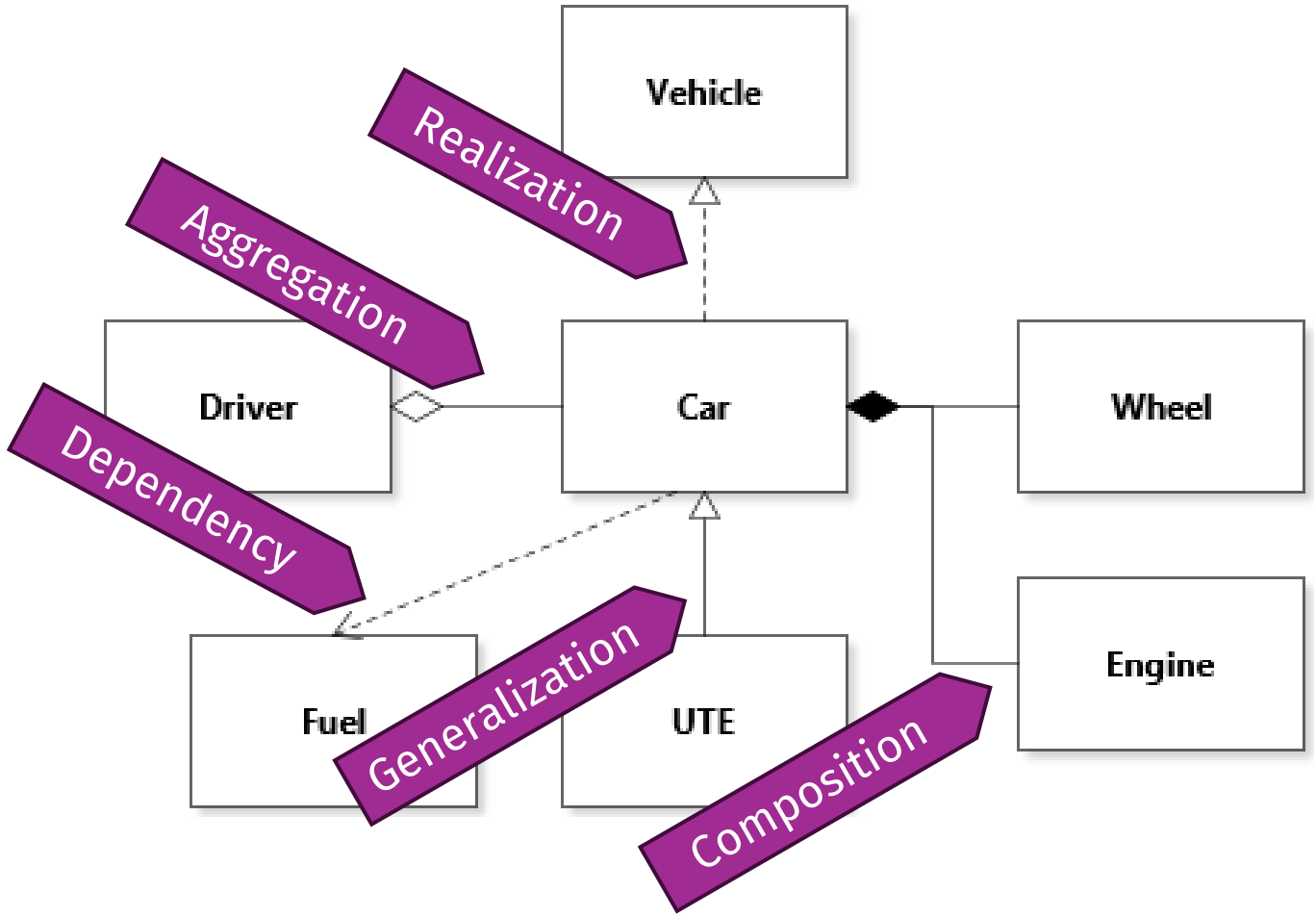
Fuel

UTE

Engine

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

UML Class Diagrams



Dependency:
Car “uses” Fuel

Aggregation:
Driver “has” a Car

Composition:
Car “is made of” Wheels and Engine

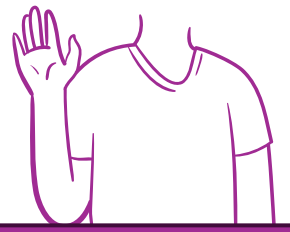
Realization:
Car “fulfils” the spec of Vehicle (interface)

Generalization:
UTE “extends” Car (inheritance)

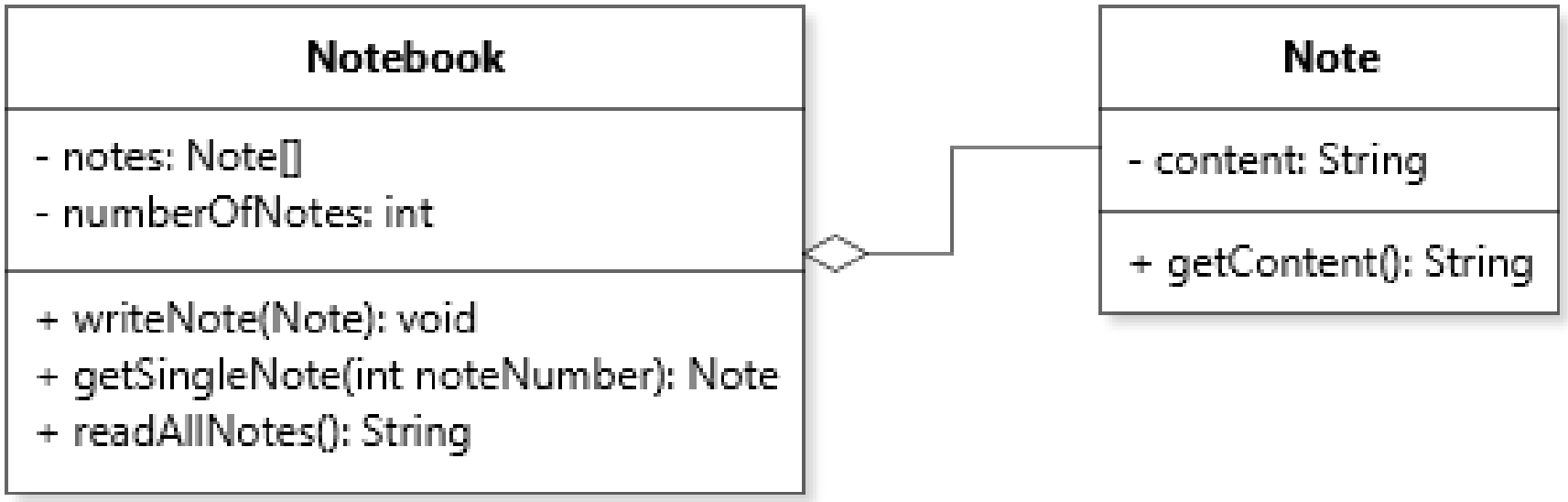
© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

UML Class Diagrams

Dependency (“uses”)
Aggregation (“has”)
Composition (“is made of”)
Realization (“fulfils”)
Generalization (“extends”)



What relationship exist between these concepts?



© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Data objects



→ Record

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Objects are collections of members (fields and methods)

Access members with '.'

```
notebook.writeNote(note);
```

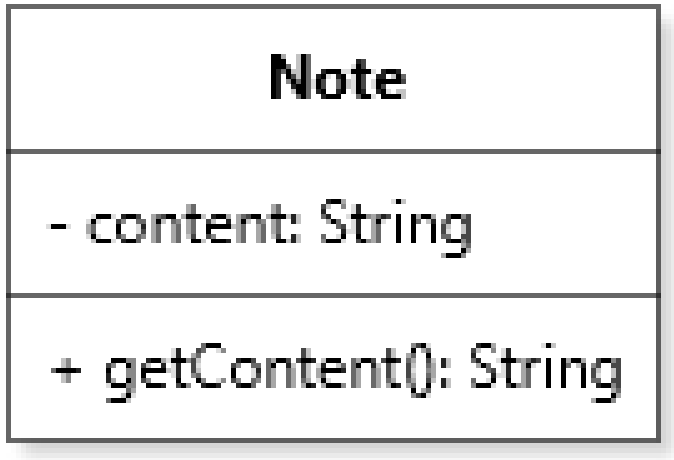
```
notebook.notes.add(note);
```

```
Notebook.addNoteToAllNotebooks(note);
```

```
System.out.println(notebook.readAllNotes());
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Some objects are little more than data holders



© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Record classes in Java

```java
public record Note(String content) {

    public Note(String... paragraphs) {
        this(String.join("\n", paragraphs));
    }

    public Note() {
        this("<Empty note>");
    }
}
```

field and method

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Read more about records: https://www.baeldung.com/java-record-keyword

Another record

```java
public class Course {

    private final String topic;
    private final int number;

    public Course(String topic, int number) {
        this.topic = topic;
        this.number = number;
    }

    public String topic() {
        return topic;
    }

    public int number() {
        return number;
    }
}
```

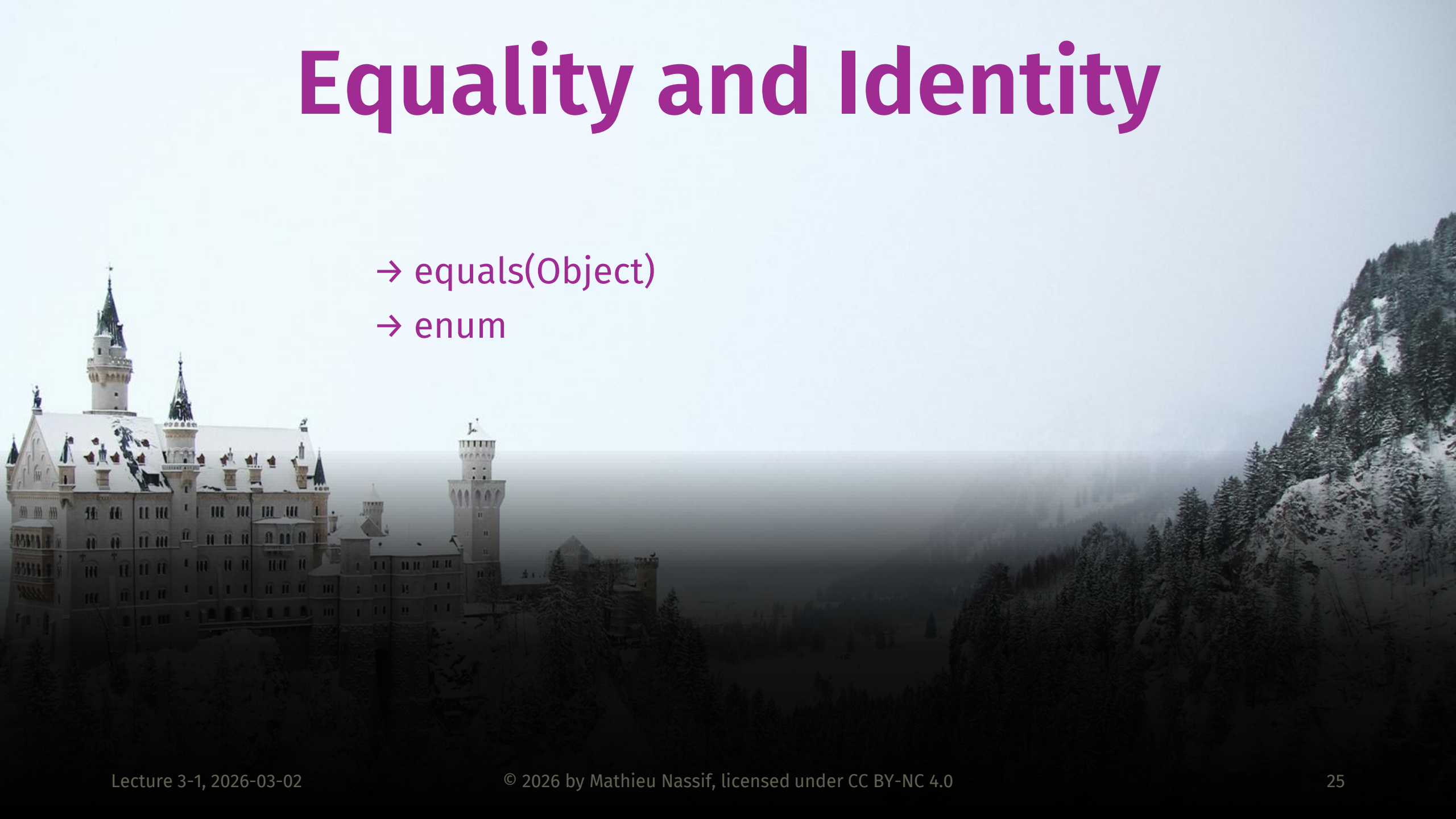
```java
public record Course(String topic, int number) {

}
```

```java
private String createCourseString() {
    String courseInfo = "Course "
            + course.topic()
            + course.number();
    return courseInfo;
}
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Equality and Identity



→ equals(Object)

→ enum

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Let’s avoid duplicate notes

```
public void writeNoteIfNoteDuplicated(Note newNote) {
    for (Note existingNote : notes) {
        if (existingNote == newNote) {
            return;
        }
    }
    writeNote(newNote);
}
```

exit early if match

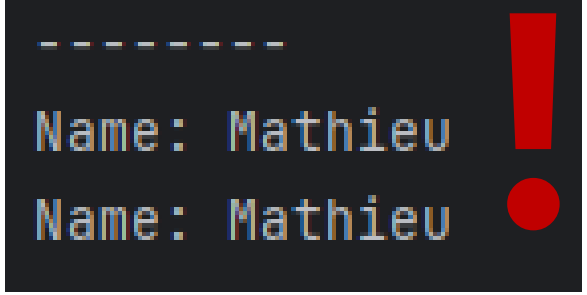
else add note

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Let’s avoid duplicate notes

```java
void main() {
    Notebook notebook = new Notebook();
    Note note = Note.createNote("Name: Mathieu");
    Note copy = Note.createNote("Name: Mathieu");
    notebook.writeNote(note);
    notebook.writeNoteIfNoteDuplicated(copy);
    System.out.println(notebook.readAllNotes());
}
```

```
Notebook
--------
Name: Mathieu
Name: Mathieu
```



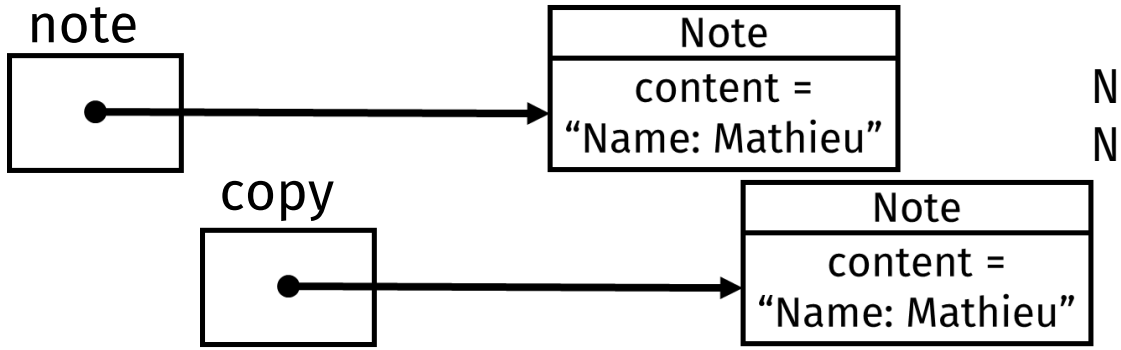
© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Equality vs Identity

for reference types

Equality

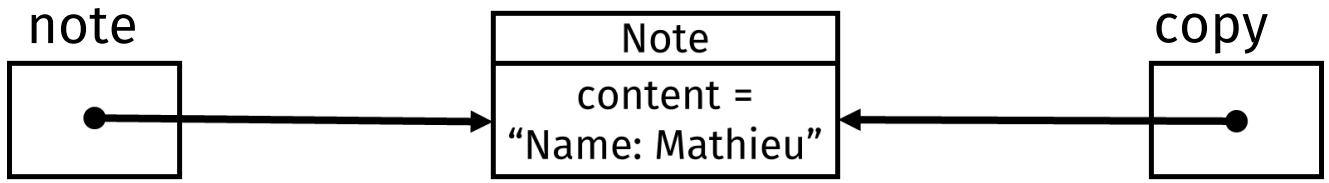
The two objects contain the same information.




```
Note note = Note.createNote(…);
Note copy = Note.createNote(…);
```

Identity

The two objects are the same (space in memory).

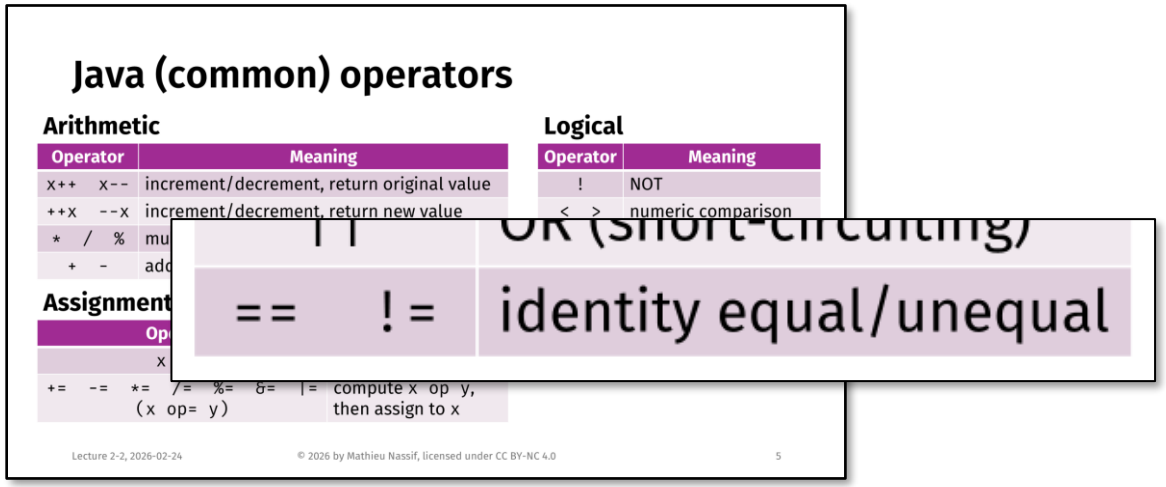



```
Note note = Note.createNote(…);
Note copy = note;
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Identity check

for reference types




```
for (Note existingNote : notes) {
    if (existingNote == newNote) {
        return;
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Equality check

for reference types

```
public void writeNoteIfNoteDuplicated(Note newNote) {
    for (Note existingNote : notes) {
        if (existingNote.equals(newNote)) {
            return;
        }
    }
    writeNote(newNote);
}
```



How does Java know how to compare equality?

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

equals(Object) method

All objects have it...

...but the default implementation is to check identity

Note

Records have a different default: compare based on content.

Notebook

```java
@Override
public boolean equals(Object o) {
    if (o == null || getClass() != o.getClass()) return false;
    Notebook notebook = (Notebook) o;
    return Objects.equals(notes, notebook.notes);
}
```

check same class

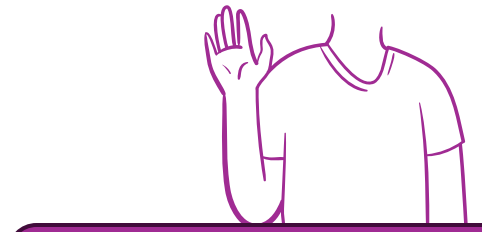
define equality

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

equals(Object) method

- Must return a `boolean`
- Must have a single `Object` parameter
- Must also implement `public int hashCode() { … }`
- Watch for `null`
  - `null.equals(object)` throws an error
  - `nonNullObject.equals(null)` should return false

- IDEs can auto-generate them



How to check “not equal”?

! x.equals(y)

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Uniqueness

Unique objects are equal if and only if they are the same (identity).

Java enums define a set of unique values.

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Enums

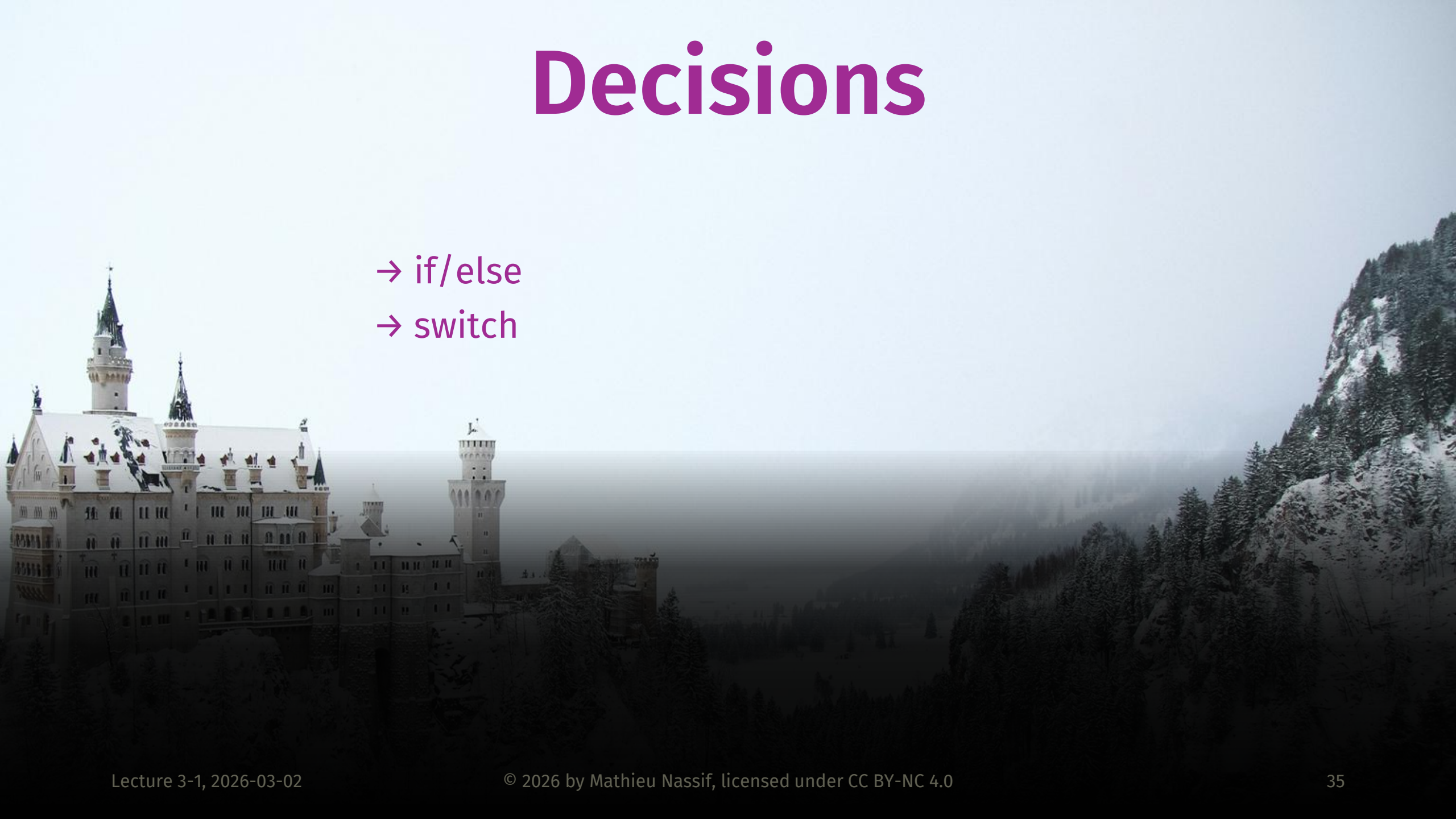
```java
public enum Topic {
    COSC, SENG, DATA, MATH
}
```

```java
new Course(Topic.SENG, 201);
```

```java
// equivalent
Topic.SENG == seng201.topic();
Topic.SENG.equals(seng201.topic());
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Decisions



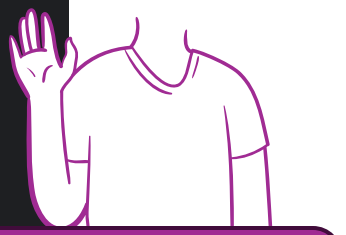
→ if/else

→ switch

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

If, else, else if

```java
private String createCourseString() {
    String courseInfo = "Course " + course.topic() + course.number();
    if (course.topic() == Topic.SENG) {
        courseInfo += " (Yay!)";
    }
    else if (course.topic() == Topic.COSC) {
        courseInfo += " (Ok)";
    }
    else if (course.topic() == Topic.MATH) {
        courseInfo += " (oh no!)";
    }
    else {
        courseInfo += " (what is that!?)";
    }
    return courseInfo;
}
```



What's the problem here?

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Switch

switch statement

```java
private String createCourseString() {
    String courseInfo = "Course " + course.topic() + course.number();
    switch (course.topic()) {
        case SENG:
            courseInfo += " (Yay!)";
            break;
        case COSC:
        case DATA:
            courseInfo += " (Ok)";
            break;
        case MATH:
            courseInfo += " (oh no!)";
            break;
        default:
            courseInfo += " (what is that!?)";
            break;
    }
    return courseInfo;
}
```

switch expression

```java
private String createCourseString() {
    String courseInfo = "Course " + course.topic() + course.number();
    courseInfo += switch (course.topic()) {
        case SENG -> " (Yay!)";
        case COSC, DATA -> " (Ok)";
        case MATH -> " (oh no!)";
        default -> " (IMPOSSIBLE!!)";
    };
    return courseInfo;
}
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Switch

Works only with `byte`, `short`, `char`, `int`, `String`, enums

```java
int x = 1 + 2;
switch (x) {
    case 0:
        System.out.println("zero");
        break;
    case 1:
    case 2:
    case 3:
        System.out.println("small");
        break;
    default:
        System.out.println("large (or negative)");
}
```

```java
String number = "zero";
int x = switch (number) {
    case "zero" -> 0;
    case "one" -> 1;
    case "two" -> 2;
    case "three" -> 3;
    default -> Integer.MAX_VALUE;
};
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

RECAP

→ static class members

→ packages, import

→ UML class dependencies

→ Record

→ Equality and identity

→ Uniqueness and enums

→ if/else and switch



© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0